MÓDULO 25

Escolhendo Linguagem e Frameworks

Como eu montaria a estratégia de testes para esse cenário

Cenário analisado: Web em React, APIs de terceiros, backend em Java e .NET e equipes com React Native.

Talyta Moysa

Engenharia de Qualidade de Software • EBAC

1. Como eu pensei a estratégia

Antes de escolher as ferramentas, eu olharia primeiro para duas coisas: as tecnologias que já existem no projeto e a facilidade de manutenção para os QAs. Como temos React/React Native no frontend, APIs de terceiros e dois backends diferentes (Java e .NET), eu evitaria criar uma estratégia em que cada camada dependesse de uma linguagem diferente.

Minha ideia seria manter JavaScript/TypeScript como base da automação que é responsabilidade de QA e deixar os testes unitários mais próximos do código de cada backend. Assim, não obrigo o time de QA a manter Java e .NET só para conseguir testar API ou interface.

Um detalhe do enunciado que eu levaria em conta

Apesar de o contexto citar uma plataforma Web, existem quatro equipes com desenvolvimento em React Native. Por isso eu trataria Web como o foco principal e mobile como uma frente complementar, principalmente para os fluxos críticos.

2. O que eu escolheria

Linguagem principal: JavaScript/TypeScript. Eu usaria essa base porque conversa bem com React e React Native e permite usar ferramentas diferentes sem mudar de linguagem a toda hora. Também facilita quando um dev frontend precisa entender ou ajudar a investigar um teste.

- É uma linguagem próxima do stack de frontend já existente.
- Atende Web, API, componentes e parte do ecossistema mobile.
- Facilita reaproveitar padrões entre os times.
- Evita que a automação de QA fique acoplada ao fato de um serviço ser Java ou .NET.

3. Ferramentas que eu usaria por camada

Camada	Ferramenta	Onde eu usaria	Por quê
Web E2E	Playwright	Principais jornadas e regressão crítica	Tem boa cobertura de navegadores, paralelismo e facilita gerar evidências.
Mobile E2E	Maestro	Fluxos críticos das aplicações React Native	A sintaxe é simples e a manutenção tende a ser mais acessível para QA.
API	PactumJS	Serviços internos e integrações	Consigo validar status, payload, schema e regras sem depender da interface.

Camada	Ferramenta	Onde eu usaria	Por quê
Contrato	PactumJS / Pactum Flow	Integrações entre sistemas	Ajuda a pegar quebra de contrato antes de chegar no E2E.
React	Jest + React Testing Library	Unitários e componentes	Dá feedback rápido e foca no comportamento do componente.
Java	JUnit 5 + Mockito	Unitários do backend Java	Faz sentido manter esses testes perto do código que está sendo desenvolvido.
.NET	xUnit + Moq	Unitários do backend .NET	Mesma ideia do Java: teste unitário junto da tecnologia nativa do serviço.



Figura 1 — Como eu distribuiria a cobertura entre as camadas.

4. Como eu distribuiria os testes

4.1 Unitários e componentes

Eu deixaria a maior parte dos testes rápidos nas camadas mais baixas. No React, usaria Jest e React Testing Library. No backend, manteria os unitários em JUnit/Mockito para Java e xUnit/Moq para .NET. Isso ajuda a encontrar defeitos mais perto da origem e sem precisar subir a aplicação inteira.

4.2 API

Como o cenário tem integrações com terceiros, eu daria bastante peso para API. Com PactumJS eu cobriria autenticação, status HTTP, regras de negócio, payloads válidos e inválidos, schemas e mensagens de erro. Essa camada seria importante também para separar se um problema está no backend ou só aparecendo na interface.

4.3 Contrato

Eu colocaria testes de contrato principalmente nas integrações que dependem de outros serviços. A ideia seria detectar cedo quando um campo muda de nome, tipo ou deixa de ser enviado. Para esse cenário, faz sentido usar Pactum Flow junto dos testes de API.

4.4 Web E2E

No Playwright eu não tentaria automatizar tudo. Eu priorizaria login, permissões, jornadas críticas e fluxos que realmente dependem da aplicação completa. Quanto mais coisa eu conseguir validar antes em componente ou API, menos a suíte E2E fica lenta e difícil de manter.

4.5 Mobile

Para React Native, eu usaria Maestro nos fluxos mais importantes. Também aqui eu evitaria uma suíte enorme: o foco seria garantir as jornadas que têm mais risco ou impacto para o usuário e deixar validações menores nas outras camadas.

> **O que eu evitaria**
>
> Eu não concentraria a automação inteira em E2E. Além de ficar mais lenta, uma suíte assim costuma ser mais difícil de manter e demora mais para mostrar onde o defeito realmente está.

5. Exemplos práticos

Web — Playwright	**API — PactumJS**
Login → busca → seleção → ação principal → confirmação. Eu usaria como fluxo crítico de ponta a ponta.	POST/PUT/DELETE + autenticação + schema + cenários de erro. Validação sem depender da tela.

Contrato Consumidor espera campos e tipos definidos. Se o contrato mudar, o teste deve acusar antes do E2E.	**Mobile — Maestro** Abrir app → autenticar → acessar conteúdo → concluir ação. Poucos fluxos, mas de alto valor.

6. Como eu colocaria isso no CI/CD

Eu deixaria os testes mais rápidos rodando em pull requests e usaria etapas posteriores do pipeline para suítes mais completas. A ideia é dar retorno rápido para quem está desenvolvendo sem abrir mão de uma validação mais ampla antes de uma liberação.

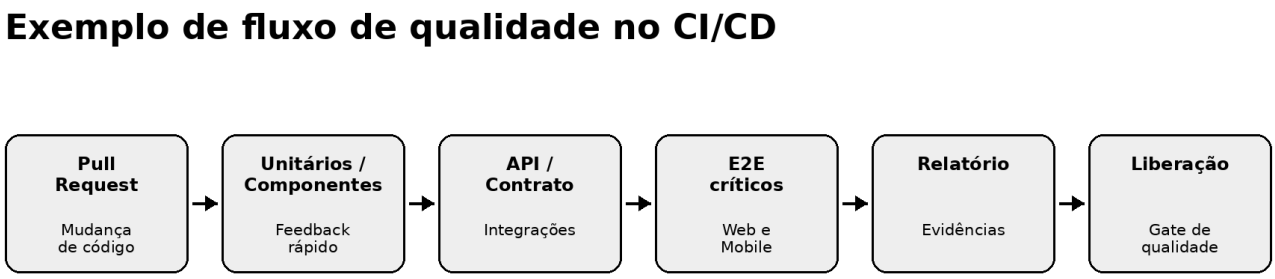


Figura 2 — Um exemplo simples de como eu distribuiria as validações no pipeline.

7. Um ponto importante para os times

Como existem várias equipes e só um Product Owner e um Scrum Master compartilhados, eu tentaria evitar que cada time criasse um padrão próprio de automação. Para mim, a padronização seria tão importante quanto a escolha da ferramenta.

- Mesma estrutura de projeto para testes Web e API.
- Convenção de nomes e organização dos cenários.
- Separação clara entre smoke, regressão e fluxos críticos.
- Revisão de código de automação entre QAs quando possível.
- Relatórios e evidências fáceis de consultar pelos times.

8. Conclusão

Para esse cenário, eu escolheria JavaScript/TypeScript como base da automação de QA, Playwright para Web, PactumJS e Pactum Flow para API e contrato, Jest/React Testing Library para React e Maestro para React Native. Nos backends, manteria os testes unitários nas próprias tecnologias de cada serviço.

O principal motivo dessa combinação é não depender de uma única camada de teste. Eu prefiro distribuir a cobertura e usar E2E só onde ele realmente agrega, porque isso tende a deixar a execução mais rápida, a manutenção mais simples e a investigação de falhas mais objetiva.

Resumo da minha escolha

JavaScript/TypeScript como base • Playwright para Web • PactumJS/Pactum Flow para API e contrato • Jest/React Testing Library para React • Maestro para mobile • JUnit/xUnit nos backends • execução integrada ao CI/CD.